

PAX Documentation v1.7.0

Portable Audit eXporter (PAX) -	
Purview Audit Log Processor	
Table of Contents	
Overview	
What It Does	
Key Features	
Intelligent Query Management	
Data Processing & Output	
Performance Optimization	
Operational Excellence	
Use Cases	
Adoption & Usage Analytics	
Compliance & Governance	
Performance & Capacity Planning	
Data Integration & BI	
Development & Testing	
Prerequisites	
Permission Details	
Why PowerShell 7+?	
Installation & Setup	
Download the Script	
First Run (Quick Start)	
Parameters Reference	
Date & Time Parameters	
Output & File Parameters	
Authentication Parameters	
Query Behavior Parameters	
Data Processing Parameters	
Offline Replay Parameters	
Parallel Execution Parameters (PowerShell 7+ only)	
Advanced Tuning Parameters	
Helper Parameters	
Authentication Methods	
1. WebLogin (Default)	
2. DeviceCode	
3. Credential	
4. Silent	
Usage Examples	
Basic Queries	
Exploded Schema Queries	
Performance Tuning	
Parallel Execution (PowerShell 7+ only)	
Offline Replay	
Agent Filtering (Live & Replay)	
Authentication Variations	
Agent Filtering	
Overview	
When to Use Agent Filtering	
Agent Filtering Examples	
How Agent Filtering Works	
Agent Filtering Performance	
Agent Field Reference	
Prompt and Response Filtering	
Overview	
PromptFilter Options	
PromptFilter Examples	
How PromptFilter Works	
PromptFilter + Agent Filtering Combination	
Performance Metrics	
Output Schema	
Output Files & Schema	
Output Files	
Schema Modes	

- Activity Types Reference
 - Common High-Volume Activities
 - Common Medium-Volume Activities
 - Common Low-Volume Activities
 - Finding Available Activities
- Advanced Features
 - Adaptive Block Sizing
 - Parallel Execution (PowerShell 7+)
 - Offline Replay Mode
 - Progress Tracking System
- Performance Tuning
 - Hitting the 10K Service Limit
 - Throttling & Rate Limiting
 - Memory Optimization
 - Parallel Execution Tuning
- Troubleshooting & FAQ
 - Common Issues
 - Frequently Asked Questions
- Known Limitations
 - Additional Notes
- Security & Compliance
 - Data Handling
 - Permissions & Least Privilege
 - Compliance Considerations
 - Security Best Practices
- License & Disclaimer
- Additional Resources
 - Microsoft Documentation
 - Related Tools

Portable Audit eXporter (PAX) - Purview Audit Log Processor

- ❑ **Quick Start:** Download the script → [PAX_Purview_Audit_Log_Processor_v1.7.0.ps1](#)
- ❑ **Release Notes:** See what's new → [release_notes/](#)
- ❑ **Documentation Archive:** Version-specific docs → [release_documentation/](#)

Script: `PAX_Purview_Audit_Log_Processor_v1.7.0.ps1`

Version: 1.7.0

Audience: IT admins, security/compliance analysts, BI/data teams

Runtime: PowerShell 5.1 (compatible) / PowerShell 7+ (recommended)

License: MIT

Table of Contents

- [1. Overview](#)
 - [2. Key Features](#)
 - [3. Use Cases](#)
 - [4. Prerequisites](#)
 - [5. Installation & Setup](#)
 - [6. Parameters Reference](#)
 - [7. Authentication Methods](#)
 - [8. Usage Examples](#)
 - [9. Agent Filtering](#)
 - [10. Prompt and Response Filtering](#)
 - [11. Output Files & Schema](#)
 - [12. Activity Types Reference](#)
 - [13. Advanced Features](#)
 - [14. Performance Tuning](#)
 - [15. Troubleshooting & FAQ](#)
 - [16. Known Limitations](#)
 - [17. Security & Compliance](#)
-

Overview

What It Does

The **Portable Audit eXporter (PAX)** is an enterprise-grade PowerShell script that exports Microsoft Purview Unified Audit Log events, with specialized support for Microsoft 365 Copilot activities and related operations. It transforms raw audit data into analysis-ready CSV files with enriched metadata, intelligent query optimization, and flexible schema options.

Core Capabilities:

- Retrieves audit events from Microsoft 365 Unified Audit Log via Exchange Online Management
- Exports to structured CSV with optional array explosion and deep JSON flattening
- Includes enriched usage & ROI fields (tokens, models, latency, acceptance metrics)
- Supports both live querying and offline replay/transformation of previously exported data
- Implements adaptive time slicing to navigate service limits intelligently
- Provides detailed logging of all operations, warnings, and performance metrics
- Automatically handles ExchangeOnlineManagement module installation and connection

Execution Modes:

1. **Standard Mode** - One row per audit record (raw JSON preserved in CopilotEventData column)
2. **Array Explosion Mode** (-ExplodeArrays) - Canonical Purview 35-column schema with array elements expanded
3. **Deep Flatten Mode** (-ExplodeDeep) - 35-column base schema + fully flattened CopilotEventData.* columns
4. **Offline Replay Mode** (-RAWInputCSV) - Re-process previously exported raw audit CSV files without querying the service
5. **Agent Filtering Mode** (-AgentsOnly or -AgentId or -ExcludeAgents) - Filter for records based on Copilot agent presence (works with live queries and replay mode)
6. **Prompt and Response Filtering Mode** (-PromptFilter) - Filter messages by isPrompt property to isolate prompts, responses, or both

[Back to Top](#)

Key Features

Intelligent Query Management

- **Adaptive Block Sizing:** Automatically adjusts time window sizes based on data density
- **10K Limit Detection:** Identifies when Microsoft 365 service cap is reached and recommends mitigation
- **Automatic Subdivision:** Binary/aggressive splitting of dense time periods to maximize completeness
- **Throttle Resilience:** Exponential backoff with jitter for retry operations
- **Volume Classification:** Smart batching based on activity type (High/Medium/Low volume)

Data Processing & Output

- **Purview Schema Compliance:** Matches Microsoft Purview's canonical exploded schema structure
- **Deep JSON Flattening:** Optional recursive flattening of nested CopilotEventData structures
- **Agent Filtering:** Filter records by specific AgentId values or any agent-related activity
- **Streaming Export:** Memory-efficient chunked CSV writing for large datasets
- **UTF-8 Encoding:** Consistent UTF-8 (no BOM) output for cross-platform compatibility
- **Header Stability:** Always writes CSV header even when zero records match (ensures schema consistency)

Performance Optimization

- **Parallel Execution (PS7+):** Concurrent processing of multiple activity types with controlled throttling
- **Learned Block Sizes:** Per-activity and global adaptive sizing based on observed densities
- **Fast CSV Writer:** Direct StreamWriter usage eliminates repeated Export-Csv overhead
- **Schema Sampling:** Configurable initial sampling to optimize column discovery vs. memory usage

Operational Excellence

- **Composite Progress:** Single weighted percentage across Query/Explosion/Export phases
- **Detailed Logging:** Comprehensive log file with parameters, decisions, warnings, and metrics
- **Automated Setup:** Detects, installs, and connects ExchangeOnlineManagement module automatically
- **Offline Replay:** Transform previously exported raw CSVs without Exchange Online connection

[□ Back to Top](#)

Use Cases

Adoption & Usage Analytics

- Track Microsoft 365 Copilot adoption across your organization
- Measure user engagement with AI features (interactions, token consumption, model usage)
- Identify power users and underutilized licenses
- Calculate ROI metrics based on time saved and acceptance rates

Compliance & Governance

- Audit Copilot interactions for regulatory compliance requirements
- Monitor data access patterns and sensitivity indicators
- Track plugin usage and custom GPT deployment
- Generate audit trails for security reviews
- Filter and analyze specific Copilot Studio declarative agent activity

Performance & Capacity Planning

- Analyze response latencies and model performance
- Identify throttling patterns and peak usage periods
- Forecast infrastructure needs based on token consumption trends
- Optimize block sizing for your tenant's data density

Data Integration & BI

- Export enriched data to Power BI, Azure Synapse, or data warehouses
- Join audit data with licensing information for coverage analysis
- Build custom dashboards with wide-schema flattened data
- Maintain historical archives with consistent schema over time

Development & Testing

- Offline replay mode for reproducible transformations
- Test schema changes against synthetic or sanitized datasets
- Validate data pipelines without querying production audit logs
- Develop downstream analytics without live tenant access

[□ Back to Top](#)

Prerequisites

Requirement	Details	Notes
PowerShell	5.1 or 7+	7+ strongly recommended for parallel execution and UTF-8
Unified Audit Logging	Enabled in tenant	Verify in Microsoft Purview compliance portal
Permissions	View-Only Audit Logs or Audit Logs role	Least privilege: Use read-only audit role
Network Access	Microsoft 365 endpoints	Ensure firewall allows connections to *.protection.outlook.com
Execution Policy	Bypass or RemoteSigned	See Authentication Methods

Note: The script automatically handles ExchangeOnlineManagement module detection, installation, and connection. No manual setup required.

Permission Details

Minimum RBAC Requirements:

- **View-Only Audit Logs** role (read-only, recommended for production)
- **Audit Logs** role (if write operations needed elsewhere)
- Member of appropriate role groups in Microsoft Purview compliance portal

Why PowerShell 7+?

Feature	PowerShell 5.1	PowerShell 7+
Parallel Execution	☐ Not Available	☐ ForEach-Object - Parallel
UTF-8 Default	☐ Requires explicit encoding	☐ Native UTF-8
Performance	Baseline	☐ 30-50% faster JSON/pipeline
TLS/Cipher Support	Legacy protocols	☐ Modern TLS 1.3
Cross-Platform	☐ Windows only	☐ Windows/macOS/Linux
Side-by-Side Install	N/A	☐ Does not replace PS 5.1

Download PowerShell 7: <https://aka.ms/powershell>

[Back to Top](#)

Installation & Setup

Download the Script

Download [PAX Purview Audit Log Processor v1.7.0.ps1](#) and save to a working directory (e.g., C:\Scripts\PAX\).

First Run (Quick Start)

```
# PowerShell 7+ (recommended)
pwsh -File .\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -StartDate 2025-10

# Windows PowerShell 5.1
powershell -ExecutionPolicy Bypass -File .\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -StartDate 2025-10
```

What Happens:

1. Script detects and installs ExchangeOnlineManagement module if needed
2. Interactive browser sign-in prompt (unless -Auth specified)
3. Queries Unified Audit Log for the specified date range
4. Exports to C:\Temp\CopilotInteraction_<timestamp>.csv (default path)
5. Creates matching .log file with detailed execution metrics

[Back to Top](#)

Parameters Reference

Date & Time Parameters

-StartDate (string)

Purpose: UTC start date (inclusive) for audit log query or replay filter

Format: yyyy-MM-dd (e.g., 2025-10-01)

Default (Live Mode): Previous full UTC day if both dates omitted

Default (Replay Mode): No filter applied if omitted

Example: -StartDate 2025-10-01

-EndDate (string)

Purpose: UTC end date (exclusive) for audit log query or replay filter

Format: yyyy-MM-dd (e.g., 2025-10-02)

Default (Live Mode): Previous full UTC day + 1 if both dates omitted

Default (Replay Mode): No filter applied if omitted

Example: -EndDate 2025-10-02

Date Behavior:

- **Live mode:** If both dates omitted, defaults to previous full UTC day (midnight to midnight)
- **Live mode:** Must specify both or neither (partial specification rejected)
- **Replay mode:** Both dates optional; act as filters on `CreationDate` column
- **Time zone:** All dates interpreted as UTC; convert local times before invocation

Output & File Parameters

-OutputFile (string)

Purpose: Target CSV file path for exported audit data

Default: C:\Temp\CopilotInteraction_<timestamp>.csv

Use When: Specifying custom output location or naming convention

Example: -OutputFile "C:\AuditData\Copilot_\$(Get-Date -Format 'yyyyMMdd').csv"

Notes: Parent directories created automatically if missing

Authentication Parameters

-Auth (string)

Purpose: Authentication method for Exchange Online connection

Valid Values: WebLogin, DeviceCode, Credential, Silent

Default: WebLogin

Use When: Automating scripts, using headless terminals, or SSO scenarios

Examples:

- -Auth WebLogin - Interactive browser sign-in (default)
- -Auth DeviceCode - Device code flow for headless/remote sessions
- -Auth Credential - Prompt for username/password (stored in memory only)
- -Auth Silent - Attempt cached token (fails if no valid token)

Not applicable in replay mode (authentication skipped when using -RAWInputCSV)

Query Behavior Parameters

-BlockHours (double)

Purpose: Initial time window size (hours) for each audit query chunk

Range: 0.016667 to 24.0 (1 minute to 24 hours)

Default: 0.5 (30 minutes)

Use When:

- Frequently hitting 10K limit (reduce to 0.25 or lower)
- Sparse historical data (increase to 2-8 hours for faster processing)
- Fine-tuning for tenant-specific data density

Examples:

- -BlockHours 0.25 - Dense periods, many records
- -BlockHours 4.0 - Sparse backfills, low activity

Notes: Script learns optimal sizes during execution; this is just the starting point

-ResultSize (int)

Purpose: Target number of records to retrieve per activity per time window

Range: 1 to 10000

Default: 10000

Use When:

- Managing memory usage (lower values = smaller batches)
- Testing with small samples
- Avoiding service throttling (reduce to 2500-5000)

Example: -ResultSize 5000

Notes: Actual results may be less; this is the requested maximum

-PacingMs (int)

Purpose: Delay (milliseconds) between paginated API calls

Range: 0 to 10000

Default: 0 (no artificial delay)

Use When:

- Experiencing frequent throttling errors
- Running during peak tenant usage
- Spreading load over time for politeness

Examples:

- -PacingMs 250 - Moderate pacing
- -PacingMs 500 - Conservative pacing

Notes: Increases total execution time proportionally

-ActivityTypes (string[])

Purpose: Array of audit log operation names to retrieve

Default: @('CopilotInteraction')

Use When:

- Querying multiple activity types in one run
- Analyzing cross-functional behaviors (Teams + Copilot)
- Comparative analysis across services

Examples:

- Single: -ActivityTypes CopilotInteraction
- Multiple: -ActivityTypes CopilotInteraction,MessageSent,FileAccessed
- Custom: -ActivityTypes @('MeetingDetail','SearchQueryPerformed')

Notes: See [Activity Types Reference](#) for common operations

-AgentId (string[])

Purpose: Filter audit records to include only those with specific AgentId value(s)

Default: Not set (no agent filtering)

Use When:

- Analyzing specific Copilot Studio declarative agents
- Tracking usage of particular agent implementations
- Filtering for known AgentId patterns

Examples:

- Single: -AgentId "CopilotStudio.Declarative.T_4e671777-fa6c-601a-b416-df08b6ae4c14.03dc0b8b-a75a-4b77-86d7-98185a176d1b"
- Multiple: -AgentId "SYSTEM_CreateGPT.declarativeCopilot", "CopilotStudio.Declarative.T_..."

Notes: - Works in both live query and replay modes - AgentId is a top-level field in AuditData JSON - Takes precedence if both -AgentId and -AgentsOnly are specified

-AgentsOnly (switch)

Purpose: Filter audit records to include only those with any AgentId present

Default: Off (no agent filtering)

Use When:

- Analyzing all agent-related activity regardless of specific agent
- Identifying records that involve Copilot Studio agents
- Filtering out non-agent Copilot interactions

Example: -AgentsOnly

Notes: - Works in both live query and replay modes - More inclusive than -AgentId (includes any record with AgentId field populated) - Combined with -ActivityTypes for refined filtering

-ExcludeAgents (switch)

Purpose: Filter audit records to EXCLUDE those with AgentId present (inverse of -AgentsOnly)

Default: Off (no agent filtering)

Use When:

- Analyzing non-agent Copilot interactions only
- Removing agent activity from analysis
- Comparing agent vs non-agent usage patterns

Example: -ExcludeAgents

Notes: - Works in both live query and replay modes - Mutually exclusive with -AgentId and -AgentsOnly - Filters at record level during parsing phase

-PromptFilter (string)

Purpose: Filter messages by Message_isPrompt property to isolate specific message types

Default: Not set (no prompt/response filtering)

Valid Values: Prompt, Response, Both, Null

Use When:

- **Prompt:** Analyzing user input patterns, query types, intent analysis
- **Response:** Extracting response content for analysis, measuring latency, tracking acceptance rates (combine with Prompt data via ThreadId for quality evaluation)
- **Both:** Full conversation analysis with defined isPrompt values
- **Null:** Debugging records with malformed or missing isPrompt properties

Examples: - -PromptFilter Prompt - Only messages where Message_isPrompt = True -
 -PromptFilter Response - Only messages where Message_isPrompt = False - -
 PromptFilter Both - Messages with either True or False (excludes nulls) - -
 PromptFilter Null - Messages with null/undefined isPrompt values

Notes: - Works in both live query and replay modes - Uses two-stage filtering: pre-filter records, then filter messages during explosion - Can be combined with -AgentsOnly, -ExcludeAgents, or -AgentId - Provides detailed metrics in summary (record/message

retention, type breakdown) - Stage 1 reduces records before explosion for performance - Stage 2 ensures clean output with no blank Message_isPrompt values

Data Processing Parameters

-ExplodeArrays (switch)

Purpose: Enable Purview canonical 35-column exploded schema (array elements → rows)

Default: Off (standard 1:1 row mode)

Use When:

- Need one row per array element for pivoting
- Matching Microsoft Purview export format
- Preparing for relational BI tools

Example: -ExplodeArrays

Notes: Forced ON automatically in replay mode

-ExplodeDeep (switch)

Purpose: Enable deep flattening (35-column base + all CopilotEventData.* columns)

Default: Off

Use When:

- Maximum data extraction for BI/ML pipelines
- Need every nested field as a separate column
- Building wide-schema data warehouses

Example: -ExplodeDeep

Notes: Significantly increases CSV width; test with short date range first

Offline Replay Parameters

-RAWInputCSV (string)

Purpose: Path to previously exported raw Purview audit CSV for offline transformation

Default: Not set (live query mode)

Use When:

- Re-processing raw exports with different explosion settings
- Development/testing without live tenant access
- Reproducible transformations for auditing

Example: -RAWInputCSV "C:\PreviousExports\Copilot_RAW_20251001.csv"

Restrictions: Cannot combine with live query parameters (-Auth, -BlockHours, -ResultSize, -PacingMs, -ParallelMode, -MaxConcurrency, -MaxParallelGroups)

Allowed with RAWInputCSV: -StartDate, -EndDate, -ActivityTypes, -AgentId, -AgentsOnly, -ExcludeAgents, -PromptFilter, -OutputFile, -ExplodeDeep, -ExportProgressInterval, -StreamingSchemaSample, -StreamingChunkSize

Parallel Execution Parameters (PowerShell 7+ only)

-ParallelMode (string)

Purpose: Control parallel execution of multiple activity types

Valid Values: Off, On, Auto

Default: Off

Use When:

- Processing multiple high-volume activity types
- Maximizing throughput on multi-core systems
- Need Auto heuristic to decide based on activity count

Examples:

- -ParallelMode Auto - Let script decide based on activity count and volume
- -ParallelMode On - Force parallel execution
- -ParallelMode Off - Sequential processing (PS 5.1 compatible)

-MaxConcurrency (int)

Purpose: Maximum concurrent threads per activity group

Range: 1 to 10

Default: 2

Use When: Fine-tuning parallel execution to avoid throttling

Example: -MaxConcurrency 4

-MaxParallelGroups (int)

Purpose: Maximum number of activity groups to process concurrently

Range: 1 to 5

Default: 3

Use When: Limiting total concurrent operations

Example: -MaxParallelGroups 2

Advanced Tuning Parameters

-StreamingSchemaSample (int)

Purpose: Number of initial records to sample for schema discovery

Range: 100 to 10000

Default: 2000

Use When:

- Wide schemas need more samples to discover all columns
- Narrow schemas can use smaller samples for faster processing

Example: -StreamingSchemaSample 5000

-StreamingChunkSize (int)

Purpose: Number of records to write per CSV flush operation

Range: 100 to 20000

Default: 5000

Use When:

- Managing memory usage (lower = more frequent flushes)
- Optimizing write performance (higher = fewer I/O operations)

Example: -StreamingChunkSize 10000

-ExportProgressInterval (int)

Purpose: Row interval for export progress updates

Range: 1 to 10000

Default: 10

Use When: Need more granular progress updates

Example: -ExportProgressInterval 5

Helper Parameters

-Help (switch)

Purpose: Display built-in help documentation

Example: .\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Help

Use When: Quick reference without opening documentation

[❏ Back to Top](#)

Authentication Methods

The script supports four authentication methods for Exchange Online:

1. WebLogin (Default)

Interactive browser-based authentication. Best for ad-hoc queries and interactive sessions.

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth WebLogin -StartDate 202
```

2. DeviceCode

Device code flow for headless/remote sessions or terminals without browser access.

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth DeviceCode -StartDate 2
```

3. Credential

Username/password prompt. Credentials stored in memory only during script execution.

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth Credential -StartDate 2
```

4. Silent

Attempts to use cached authentication token. Falls back to WebLogin if no valid token exists.

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth Silent -StartDate 2025-
```

[Back to Top](#)

Usage Examples

Basic Queries

```
# Standard mode - previous day (auto-default)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1

# Specific date range
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -StartDate 2025-10-01 -EndDat

# Custom output path
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -StartDate 2025-10-01 -EndDat

# Multiple activity types
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -StartDate 2025-10-01 -EndDat
```

Exploded Schema Queries

```
# Array explosion (35-column Purview schema)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ExplodeArrays -StartDate 202

# Deep flatten (maximum column extraction)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ExplodeDeep -StartDate 2025-
```

Performance Tuning

```
# Reduce block size for dense data (hitting 10K limit)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -BlockHours 0.25 -StartDate 2025-11-01

# Increase block size for sparse historical data
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -BlockHours 4.0 -StartDate 2025-11-01

# Add pacing to reduce throttling
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -PacingMs 250 -StartDate 2025-11-01
```

Parallel Execution (PowerShell 7+ only)

```
# Auto-detect parallel benefit
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ParallelMode Auto -ActivityType All

# Force parallel with custom concurrency
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ParallelMode On -MaxConcurrency 4
```

Offline Replay

```
# Basic replay (forced explosion)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv"

# Replay with deep flatten and date filtering
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv" -DeepFlatten -StartDate 2025-11-01

# Replay with activity filtering
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv" -ActivityType All

# Replay with agent filtering (any agent)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv" -AgentId "*"

# Replay with specific agent ID
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv" -AgentId "SYSTEM_CreateGPT.de"
```

Agent Filtering (Live & Replay)

```
# Filter for any agent-related records (Live query)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -AgentsOnly -StartDate 2025-11-01

# Filter for specific agent ID(s)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -AgentId "SYSTEM_CreateGPT.de"

# Multiple agent IDs with deep flatten
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ExplodeDeep -AgentId "SYSTEM_CreateGPT.de,SYSTEM_CreateGPT.de"

# Agent filtering in replay mode
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -RAWInputCSV "C:\PreviousExplosion\audit_log.csv" -AgentId "SYSTEM_CreateGPT.de"
```

Authentication Variations

```
# Device code for headless session
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth DeviceCode -StartDate 2025-11-01

# Credential prompt
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth Credential -StartDate 2025-11-01

# Silent (cached token)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -Auth Silent -StartDate 2025-11-01
```

[Back to Top](#)

Agent Filtering

Overview

Agent Filtering enables targeted extraction of Copilot agent-specific audit records from your audit logs. This feature is essential for enterprises analyzing AI agent usage, ROI metrics, and compliance requirements specific to Copilot agents and declarative agents.

Why Use Agent Filtering?

- **Performance:** Process only relevant records when you need agent-specific analysis (typical reduction: 99%+ of non-agent records filtered out)
- **Cost Efficiency:** Reduce data egress, storage, and processing costs by exporting only agent-related activities
- **Focused Analysis:** Streamline BI pipelines, Power BI dashboards, and ML models to analyze agent adoption, usage patterns, and ROI
- **Compliance:** Isolate agent interactions for regulatory audits, data governance, and security investigations
- **Multi-Agent Tracking:** Monitor specific declarative agents, custom agents, or Copilot Studio agents across your tenant

When to Use Agent Filtering

Use -AgentsOnly when: - You want all records that contain any AgentId (any Copilot agent activity) - Building comprehensive agent usage dashboards - Analyzing overall agent adoption across the organization - Tracking all AI agent interactions for compliance

Use -AgentId when: - You need records for specific agent(s) only (e.g., "CopilotStudio.Declarative.12345") - Troubleshooting a particular custom or declarative agent - Analyzing ROI/performance of specific agent deployments - Auditing a specific agent's interactions for security review

Agent Filtering Examples

```
# Export ALL agent-related records from Live query
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -AgentsOnly `
  -OutputFile "C:\Exports\AgentActivity.csv"

# Filter for specific AgentId (single)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -AgentId "CopilotStudio.Declarative.a1b2c3d4" `
  -OutputFile "C:\Exports\SpecificAgent.csv"

# Filter for multiple specific AgentIds
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -AgentId "CopilotStudio.Declarative.agent1","CopilotStudio.Declarative
  -OutputFile "C:\Exports\MultipleAgents.csv"

# Replay mode: Filter agents from previously exported data
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -RAWInputCSV "C:\Exports\RawAuditLogs.csv" `
  -AgentsOnly `
  -ExplodeDeep `
  -OutputFile "C:\Exports\AgentActivity_Exploded.csv"

# Replay mode: Filter specific AgentId from previously exported data
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -RAWInputCSV "C:\Exports\RawAuditLogs.csv" `
  -AgentId "CopilotStudio.Declarative.a1b2c3d4" `
  -OutputFile "C:\Exports\SpecificAgent_Replay.csv"

# Combine with deep explosion for maximum analysis detail
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -AgentsOnly `
  -ExplodeDeep `
  -OutputFile "C:\Exports\AgentActivity_DeepAnalysis.csv"
```

How Agent Filtering Works

1. **Pre-Parsing Phase:**
 - In replay mode, JSON audit data is pre-parsed for all records
 - Enables fast filtering without repeated JSON parsing
2. **Agent Filtering Phase:**
 - Each record's `ParsedAuditData.AgentId` field is evaluated
 - `-AgentsOnly`: Includes any record where `AgentId` is present and non-empty
 - `-AgentId`: Includes records where `AgentId` matches one of the specified values (case-insensitive)
 - Non-matching records are excluded from output
3. **Output Generation:**
 - Only filtered records proceed to explosion/flattening
 - Summary includes pre/post filter counts and retention rate
 - Log file documents exact filter criteria applied

Agent Filtering Performance

Live Query Mode: - Agent filtering occurs server-side via activity type selection - Use standard Copilot activity types (`CopilotInteraction`, etc.) - Agent switches apply additional post-retrieval filtering

Replay Mode: - Processes up to ~5,000 records/second during filtering - Progress bar shows pre-parsing and filtering phases separately - Example: 367,796 records → 20,240 agent records in ~80 seconds total

Memory Usage: - Low overhead: only filtered records remain in memory - Safe for processing multi-million record datasets - Ideal for post-processing large audit exports

Agent Field Reference

The `AgentId` field appears in Copilot audit records and identifies the specific agent involved in the interaction:

Common Agent Patterns: - `CopilotStudio.Declarative.<GUID>` - Declarative agents created in Copilot Studio - `CustomAgent.<name>` - Custom-built agents - Copilot-specific identifiers for built-in agents

Output Columns: - `AgentId` - The unique agent identifier - `AgentName` - Human-readable agent name (if available) - `AppIdentity` - Application context for the agent - Plus all standard Copilot usage fields

[❏ Back to Top](#)

Prompt and Response Filtering

Overview

Prompt and Response Filtering (`-PromptFilter`) enables targeted extraction of specific message types from Copilot audit logs based on the `Message_isPrompt` property. This feature is essential for analyzing prompt engineering, conversation patterns, and user interaction behaviors.

Why Use PromptFilter?

- **Prompt Analysis:** Isolate user prompts to analyze query patterns, intent, and demand
- **Response Analysis:** Extract Copilot responses for content analysis, latency measurement, and tracking acceptance rates (combine with prompts via `ThreadId` for full conversation context)
- **Conversation Segmentation:** Separate prompts from responses for training data or analysis pipelines
- **Data Reduction:** Reduce output size by 50%+ when only prompts or responses are needed
- **Performance:** Two-stage filtering optimizes processing (pre-filter records + message-level filtering)

PromptFilter Options

Option	Description	Message_isPrompt Value	Use Case
Prompt	Only prompts (user inputs)	True	Analyze what users are asking
Response	Only responses (Copilot outputs)	False	Extract response content (combine with prompts via ThreadId for quality evaluation)
Both	Both prompts and responses	True or False	Full conversation analysis
Null	Messages with no isPrompt value	null or empty	Debug malformed data

PromptFilter Examples

```
# Export only user prompts
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -ExplodeArrays `
  -PromptFilter Prompt `
  -OutputFile "C:\Exports\UserPrompts.csv"

# Export only Copilot responses
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -ExplodeArrays `
  -PromptFilter Response `
  -OutputFile "C:\Exports\CopilotResponses.csv"

# Combine with agent filtering: Agent prompts only
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -ExplodeArrays `
  -AgentsOnly `
  -PromptFilter Prompt `
  -OutputFile "C:\Exports\AgentPrompts.csv"

# Replay mode: Filter prompts from previous export
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -RAWInputCSV "C:\Exports\RawAuditLogs.csv" `
  -PromptFilter Prompt `
  -OutputFile "C:\Exports\PromptsOnly.csv"
```

How PromptFilter Works

Two-Stage Filtering for Optimal Performance:

- Stage 1 (Pre-Filter):** Record-level filtering BEFORE explosion
 - Always applies** when PromptFilter is used
 - Analyzes each record's Messages array
 - Categorizes records: Mixed (prompts+responses), Prompt-only, Response-only, No messages
 - Filters out records without matching messages
 - Typical reduction: 10-15% of records filtered before explosion
 - In non-explosion mode (live query without explosion switches), this is the only filtering stage
- Stage 2 (Message-Level):** Prompt/response filtering DURING explosion
 - Only applies during explosion** (when using -ExplodeArrays, -ExplodeDeep, or -RAWInputCSV replay mode)
 - Filters individual messages within each record
 - Only outputs rows for messages matching the filter

- Prevents blank Message_isPrompt values in output
- Not used in standard 1:1 mode (live query without explosion switches)

PromptFilter Behavior by Option:

- **Prompt:** Stage 1 keeps records with at least one prompt; Stage 2 (if explosion enabled) outputs only prompt messages
- **Response:** Stage 1 keeps records with at least one response; Stage 2 (if explosion enabled) outputs only response messages
- **Both:** Stage 1 keeps all records with messages; Stage 2 (if explosion enabled) outputs messages with defined isPrompt values
- **Null:** Stage 1 keeps records with null isPrompt messages; Stage 2 (if explosion enabled) outputs only messages with null isPrompt

PromptFilter + Agent Filtering Combination

PromptFilter works independently with all agent switches:

```
# Agent interactions only, prompts only
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -ExplodeArrays `
  -AgentsOnly `
  -PromptFilter Prompt `
  -OutputFile "C:\Exports\AgentPrompts.csv"

# Non-agent interactions only, prompts only
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -StartDate 2025-10-01 `
  -EndDate 2025-10-02 `
  -ExplodeArrays `
  -ExcludeAgents `
  -PromptFilter Prompt `
  -OutputFile "C:\Exports\NonAgentPrompts.csv"
```

Performance Metrics

The script provides detailed PromptFilter metrics in the summary:

- **Record-level:** Records before/after filter, retention rate
- **Record type breakdown:** Mixed, Prompt-only, Response-only, No messages (with percentages)
- **Message-level:** Messages before/after filter, retention rate
- **Processing time:** Stage 1 pre-filter execution time

Output Schema

When using PromptFilter with -ExplodeArrays or -ExplodeDeep, the Message_isPrompt column will contain:

- **PromptFilter=Prompt:** All rows have Message_isPrompt = True
- **PromptFilter=Response:** All rows have Message_isPrompt = False
- **PromptFilter=Both:** Mix of True and False values
- **PromptFilter=Null:** All rows have blank Message_isPrompt values

[□ Back to Top](#)

Output Files & Schema

Output Files

Every execution produces two files:

1. CSV File (Data Export)

- **Location:** Specified by -OutputFile parameter (default: C:\Temp\CopilotInteraction_<timestamp>.csv)

- **Encoding:** UTF-8 without BOM
- **Format:** Standard CSV with quoted fields
- **Header:** Always written (even when zero records match)
- **Line Endings:** CRLF on Windows, LF on macOS/Linux

2. Log File (Execution Metrics)

- **Location:** Same directory as CSV, .csv replaced with .log
- **Contains:**
 - Script parameters and version
 - Authentication method and connection details
 - Query plan and adaptive block sizing decisions
 - Progress updates and phase transitions
 - Warnings (10K limits, throttling, schema changes)
 - Final metrics (records processed, time elapsed, throughput)

Schema Modes

Standard Mode (Default)

One row per audit record. CopilotEventData preserved as JSON string in a single column.

Column Count: Variable (base audit fields + CopilotEventData JSON column)

Use When: Need raw data for custom processing or minimal transformation

Exploded Mode (-ExplodeArrays)

Purview canonical 35-column schema. Array elements (Messages, AccessedResources, AISystemPlugins) expanded to separate rows.

Column Count: 35 base columns

Base Columns (35): 1. RecordId 2. CreationDate 3. RecordType 4. Operation 5. UserId 6. AssociatedAdminUnits 7. AssociatedAdminUnitsNames 8. AgentId 9. AgentName 10. AppIdentity_AppId 11. AppIdentity_DisplayName 12. AppIdentity_PublisherId 13. ApplicationName 14. CreationTime 15. ClientRegion 16. Audit_UserId 17. AppHost 18. ThreadId 19. Context_Id 20. Context_Type 21. Message_Id 22. Message_isPrompt 23. AccessedResource_Action 24. AccessedResource_PolicyDetails 25. AccessedResource_SiteUrl 26. AISystemPlugin_Id 27. AISystemPlugin_Name 28. ModelTransparencyDetails_ModelName 29. MessageIds 30. Message_AcceptanceState 31. Message_TokenCount 32. Message_CharacterCount 33. Message_LengthPreference 34. Message_LatencyMilliseconds 35. ExplosionTruncated

Use When: Need relational format for BI tools or matching Microsoft Purview exports

Deep Flatten Mode (-ExplodeDeep)

35 base columns + all nested CopilotEventData.* columns. Maximum data extraction with every nested field as a separate column.

Column Count: 35+ (dynamic based on data)

Use When: - Maximum data extraction for BI/ML pipelines - Need every nested field accessible as a column - Building wide-schema data warehouses

Warning: Significantly increases CSV width and processing time. Test with short date range first.

Activity Types Reference

Common High-Volume Activities

- CopilotInteraction - Microsoft 365 Copilot usage events
- MessageSent - Teams/Exchange message sending
- FileAccessed - SharePoint/OneDrive file access

- MailItemsAccessed - Email access events

Common Medium-Volume Activities

- MessageRead - Message read receipts
- FileModified - File edit operations
- MeetingDetail - Teams meeting metadata
- SearchQueryPerformed - Search queries

Common Low-Volume Activities

- CreatePlugin - Copilot plugin creation
- UpdatePlugin - Plugin modifications
- DeletePlugin - Plugin removal
- EnablePlugin / DisablePlugin - Plugin state changes

Finding Available Activities

For a complete list of available Purview audit activities and operations, refer to the Microsoft Learn documentation:

[□ Audit log activities - Microsoft Purview](#)

This comprehensive reference includes all available operations across Microsoft 365 services, including SharePoint, Exchange, Teams, Copilot, and more.

[□ Back to Top](#)

Advanced Features

Adaptive Block Sizing

The script automatically adjusts time window sizes based on observed data density:

- **Initial Block:** Starts with -BlockHours parameter (default 30 minutes)
- **Learning Phase:** Monitors record counts per window
- **Automatic Subdivision:** Splits windows hitting the 10K service limit
- **Progressive Refinement:** Shrinks blocks for dense periods, expands for sparse periods
- **Per-Activity Learning:** Maintains separate learned block sizes for each activity type

Parallel Execution (PowerShell 7+)

When processing multiple activity types, parallel execution can significantly improve performance:

- **Auto Mode:** Script heuristically determines if parallel execution will benefit
- **Forced Mode:** Always use parallel execution regardless of activity count
- **Throttling Control:** Configurable concurrency limits to avoid overwhelming the service
- **Group Processing:** Activities classified by volume (High/Medium/Low) and processed in batches

Offline Replay Mode

Re-process previously exported raw audit CSV files without querying Exchange Online:

- **No Authentication Required:** Skip connection to Microsoft 365
- **Flexible Filtering:** Apply date, activity, and agent filters to existing data
- **Schema Transformation:** Convert raw exports to exploded or deep flatten schemas
- **Reproducible Analysis:** Test transformations against known datasets
- **Development Workflow:** Build pipelines without production access

Progress Tracking System

Real-time progress updates across three phases:

Display Format:

PAX Purview Audit Log Processing
Status: Query: 45/100(45%) | Explosion: 12000/25000(48%) | Export: 0/1(0%)
:: 42%

Components:

- **Overall percentage:** Weighted composite across all phases
- **Phase detail:** Current/Total (percentage) for each active phase
- **Batch info:** Current batch number, estimated total, percentage range
- **Record range:** Shows which records currently processing (in batches)

[Back to Top](#)

Performance Tuning

Hitting the 10K Service Limit

Symptoms:

- Log shows: CRITICAL: 10K limit reached for time window <dates>
- CSV may be incomplete for dense periods

Immediate Action:

```
# Reduce block hours to 15 minutes or Less
pwsh -File .\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 `
  -BlockHours 0.25 `
  -StartDate 2025-10-03 `
  -EndDate 2025-10-03
```

Progressive Tuning:

1. Start: -BlockHours 0.5 (30 min) → If still hitting: 0.25 (15 min)
2. If still saturated: 0.133333 (8 min) → 0.066667 (4 min)
3. Minimum: 0.016667 (1 min)

Verification:

- Check log for “Data retrieval completed without hitting limits”
- Compare record counts across runs
- Monitor Hit10KLimit flag in metrics section

Throttling & Rate Limiting

Symptoms:

- Log shows: WARNING: Throttling detected, backing off...
- Frequent retry attempts
- Extended execution times

Solutions:

```
# Add inter-page pacing (250ms delay between API calls)
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -PacingMs 250 -StartDate 2025

# Reduce ResultSize to smaller batches
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ResultSize 5000 -StartDate 2

# Combine both approaches
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ResultSize 5000 -PacingMs 25
```

Memory Optimization

For Deep Flatten with Wide Schemas:

```
# Increase schema sample, reduce chunk size
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ExplodeDeep `
-StreamingSchemaSample 5000 `
-StreamingChunkSize 2000 `
-StartDate 2025-10-01 `
-EndDate 2025-10-02
```

For Narrow Schemas (Faster Processing):

```
# Reduce schema sample, increase chunk size
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ExplodeArrays `
-StreamingSchemaSample 1000 `
-StreamingChunkSize 10000 `
-StartDate 2025-10-01 `
-EndDate 2025-10-02
```

Parallel Execution Tuning

Conservative Approach (Avoid Throttling):

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ParallelMode On `
-MaxConcurrency 2 `
-MaxParallelGroups 2 `
-ActivityTypes CopilotInteraction,MessageSent,FileAccessed
```

Aggressive Approach (Maximum Throughput):

```
.\PAX_Purview_Audit_Log_Processor_v1.7.0.ps1 -ParallelMode On `
-MaxConcurrency 4 `
-MaxParallelGroups 3 `
-ActivityTypes CopilotInteraction,MessageSent,FileAccessed
```

[❏ Back to Top](#)

Troubleshooting & FAQ

Common Issues:

- [Authentication Failures](#)
- [No Data Returned](#)
- [10K Limit Warnings](#)
- [Memory Issues](#)
- [Throttling Errors](#)

Frequently Asked Questions:

- [Does the script modify any data?](#)
 - [What timezone are dates in?](#)
 - [Can I filter by specific users or models at the source?](#)
 - [How deep does the script flatten JSON?](#)
 - [Can I run this in an automated schedule?](#)
 - [What if I need older audit logs?](#)
 - [Does the script work on macOS/Linux?](#)
 - [How do I handle very large date ranges?](#)
 - [Can I customize the output schema?](#)
 - [What's the difference between -ExplodeArrays and -ExplodeDeep?](#)
-

Common Issues

Authentication Failures

Problem: “Unable to connect to Exchange Online”

Solutions:

- Verify you have View-Only Audit Logs or Audit Logs role assigned
- Check network connectivity to *.protection.outlook.com
- Try different auth method: -Auth DeviceCode for headless sessions
- Clear cached credentials: Restart PowerShell session

No Data Returned

Problem: CSV contains only header, no records

Solutions:

- Verify Unified Audit Logging is enabled in your tenant
- Check date range (dates are UTC, not local time)
- Confirm activity type spelling: `-ActivityTypes CopilotInteraction` (case-sensitive)
- Verify users have generated audit events in the date range
- Check audit log retention period (default 90 days)

10K Limit Warnings

Problem: Log shows “CRITICAL: 10K limit reached”

Solutions:

- Reduce `-BlockHours` parameter (try 0.25 or 0.133333)
- Run script multiple times with shorter date ranges
- Check adaptive subdivision is working (log should show automatic splits)
- Consider if data is genuinely dense (may need multiple runs)

Memory Issues

Problem: Script consumes excessive memory or crashes

Solutions:

- Reduce `-StreamingChunkSize` (try 2000 or 1000)
- Increase `-StreamingSchemaSample` to discover schema earlier (try 5000)
- Avoid `-ExplodeDeep` for initial runs (use `-ExplodeArrays` instead)
- Process shorter date ranges
- Close other applications to free memory

Throttling Errors

Problem: Frequent “Throttling detected” messages

Solutions:

- Add pacing: `-PacingMs 250` or `-PacingMs 500`
- Reduce `ResultSize`: `-ResultSize 5000`
- Run during off-peak hours
- Disable parallel mode if enabled
- Consider if tenant is under heavy load

Frequently Asked Questions

Q: Does the script modify any data?

A: No. The script is read-only and only exports audit data. No modifications are made to audit logs or tenant configuration.

Q: What timezone are dates in?

A: All dates are interpreted as UTC. Output timestamps are also UTC in ISO 8601 format (`yyyy-MM-ddTHH:mm:ss.fffZ`).

Q: Can I filter by specific users or models at the source?

A: No, filtering happens after export. The Unified Audit Log API does not support user/model filtering. Export data and filter in post-processing or BI tools.

Q: How deep does the script flatten JSON?

A: Standard explode: 60 levels. Deep flatten: 120 levels. JSON serialization: 60 levels.

These are constants in the script and can be adjusted if needed.

Q: Can I run this in an automated schedule?

A: Yes. Use `-Auth Silent` with cached credentials or `-Auth Credential` with saved credentials. Consider using Task Scheduler (Windows) or cron (macOS/Linux).

Q: What if I need older audit logs?

A: Audit retention depends on your tenant's licensing. E3/E5 licenses retain 90-365 days. Check Microsoft Purview compliance portal for your retention period.

Q: Does the script work on macOS/Linux?

A: Yes, with PowerShell 7+. Install PowerShell 7 and ExchangeOnlineManagement module. Authentication methods may vary (WebLogin, DeviceCode recommended).

Q: How do I handle very large date ranges?

A: Break into smaller chunks (weekly or monthly), run separately, then concatenate CSV files. Use `-OutputFile` to name by date range.

Q: Can I customize the output schema?

A: The 35-column base schema is fixed to match Purview standards. In `-ExplodeDeep` mode, additional columns are auto-discovered from nested data.

Q: What's the difference between `-ExplodeArrays` and `-ExplodeDeep`?

A: `-ExplodeArrays` creates 35 columns with array elements as separate rows. `-ExplodeDeep` adds all nested `CopilotEventData.*` fields as additional columns (wide schema).

[❏ Back to Top](#)

Known Limitations

Area	Limitation / Behavior	Mitigation / Guidance
Unified Audit 10K cap	Each Search-UnifiedAuditLog window tops at 10,000 records	Script auto-subdivides; if still saturated, re-run with smaller <code>-BlockHours</code> ($\leq 30m$)
Row explosion cap	Per original record explosion capped at 1,000 rows (<code>ExplosionTruncated</code> flag)	Investigate fan-out; consider narrower date, filter operations, or deep analysis separately
JSON / flatten depth	JSON serialization depth fixed at 60; deep flatten recursion capped at 120	Extremely deep structures beyond caps truncated; adjust constants if required
Memory usage	Streaming, chunked export by default	Tune with <code>-StreamingSchemaSample</code> / <code>-StreamingChunkSize</code> ; shard by date for extreme spans
Replay mode	Non-exploded mode disabled; always at least exploded schema	Use live mode if raw 1:1 row shape required
Parallel mode	Only helps multi-activity sets; single high-volume activity remains serial	Add more activity types or accept serial path
Time zones	Dates interpreted as UTC; <code>yyyy-MM-dd</code> must be UTC	Convert local times to UTC prior to invocation to avoid DST drift
Streaming export	Always on (chunked)	Adjust sample/chunk sizes for schema width & memory balance

Additional Notes

Streaming Export Behavior:

- Samples initial records (default 2000) to finalize column schema
- Writes header once, then processes rows in chunks (default 5000)
- Auto-adjusts chunk size based on column count (>250/500/750/1000 columns → smaller chunks)
- Boosts chunk size for narrow schemas (≤60 columns → up to 15K)
- New columns discovered after schema freeze are ignored (warning emitted)

Fast CSV Writer:

- Uses in-process UTF-8 StreamWriter with manual escaping
- No repeated Export-Csv invocations
- Significantly faster for large exports (>300K rows)
- Transparent to user (no parameter required)

Timestamp Normalization:

- All timestamps output in UTC
- ISO 8601 format with millisecond precision: yyyy-MM-ddTHH:mm:ss.fffZ
- Eliminates locale ambiguity
- Simplifies downstream parsing

Parallel Replay Explosion (PS 7+ only):

- When replaying raw CSV with large record count
- Switches to controlled parallel explosion
- Dynamic throttle + batch resizing (targets ~0.8s–2.5s batches)
- Metrics emitted at completion

[❏ Back to Top](#)

Security & Compliance

Data Handling

- **Read-Only Operations:** Script never modifies audit logs or tenant configuration
- **Credential Security:** Authentication credentials never written to disk (memory only)
- **Audit Trail:** All operations logged with timestamps and parameters
- **No External Services:** No data transmitted to third-party services
- **Local Processing:** All data processing occurs on the execution machine

Permissions & Least Privilege

Recommended:

- Use **View-Only Audit Logs** role (read-only)
- Create dedicated service account for automated runs
- Limit access to output files (contain sensitive audit data)
- Review Microsoft Purview RBAC documentation

Not Required:

- No Global Administrator rights needed
- No tenant configuration changes
- No write access to audit logs

Compliance Considerations

- **Data Residency:** Export files remain on execution machine (control geography)
- **Retention:** Manage CSV retention per organizational policies
- **Encryption:** Consider encrypting output files for sensitive environments
- **Access Control:** Limit script execution to authorized personnel
- **Audit Review:** Regularly review log files for anomalies

Security Best Practices

1. **Execution Policy:** Use Bypass or RemoteSigned (avoid Unrestricted)

2. **Script Verification:** Validate script hash before execution
3. **Credential Management:** Avoid storing passwords in scripts (use `-Auth Silent` or `-Auth DeviceCode`)
4. **Network Security:** Ensure TLS 1.2+ for Exchange Online connections
5. **Output Protection:** Store CSV files in encrypted volumes or secure file shares
6. **Access Logging:** Enable filesystem auditing for output directories

[□ Back to Top](#)

License & Disclaimer

License: MIT License - see [LICENSE](#) for full text

Copyright: © Microsoft Corporation

Disclaimer: This script is provided “AS IS” without warranties or official support. Validate fit for purpose before production use. Not endorsed or officially supported by Microsoft Product Groups. Community-driven maintenance model.

[□ Back to Top](#)

Additional Resources

Microsoft Documentation

- [Microsoft Purview Audit \(Premium\)](#) - Overview of audit capabilities
- [Audit log activities](#) - Complete list of auditable activities
- [Search the audit log](#) - Audit log search basics
- [Exchange Online PowerShell](#) - Exchange Online module documentation

Related Tools

- [Power BI](#) - Visualize exported audit data
- [Azure Synapse Analytics](#) - Data warehousing for large audit datasets
- [Microsoft Sentinel](#) - SIEM integration for audit logs

[□ Back to Top](#)
